Waffle 5.1 fast (high) — PO Management System

CSE 311 & CSE 321 joint project report · 12 November 2026 · **Team:** EmreSndkc · brkznqr · eminozkanca · Berkegn · **Repo:** https://github.com/EmreSndkc/waffle_v5.1

CSE 311 — 1 Punchline & Backlog

Punchline: Waffle 5.1 is a Django web system that automatically computes every student's 11 Program Outcome (PO) scores from the grades instructors enter, giving the department an accreditation-ready PO profile.

Backlog: 16 features, sized in Fibonacci story points (M = must-have, I = innovative; 64 points in total). The backlog currently lives in this table; moving it to a GitHub Projects board as issues is the first task of S1.

#	Feature	Type	Points	Status
1	Login/logout, role-based redirect	M	3	S1
2	PO management + seed data for the 11 POs	M	2	**Done (S0)**
3	Course creation, instructor assignment, student enrollment	M	3	Admin panel only
4	LO + assessment + percentage entry screen	M	5	Admin panel only
5	LO→PO mapping matrix (1–5)	M	3	Admin panel only
6	Grade entry screen	M	3	Admin panel only
7	PO calculation engine + snapshot + automatic recalculation	M	5	**Done (S0)**
8	Student dashboard: PO chart	M	3	S3
9	Bulk grade upload (Excel/CSV)	I	5	S3
10	Extract LOs and weights from the syllabus PDF	I	8	S4
11	PDF performance report	I	5	S4
12	Department-head dashboard: PO × course heat map	I	5	S5
13	Risk alert e-mail (PO below threshold)	I	3	S5
14	"What-if" simulator	I	3	S5
15	Change history (audit log)	I	3	S5
16	REST API + Turkish/English interface	I	5	If time allows

CSE 311 — 2 Work & Velocity

Sizing: Fibonacci story points (1, 2, 3, 5, 8); points express feature difficulty. **Sprint length:** 1 week; planning at the start, review and `develop` → `main` merge at the end. **Completed sprints:** 1 (S0, closed on 11 November).

Velocity: S0 = **7 points** (#2 and #7). Only fully finished features are counted; #3–#6 are usable through the admin panel but are not counted because their role-specific screens do not exist yet. The remaining 57 points are spread over six sprints, which requires an average of 9.5 points per sprint.

Member	Code work in S0	Diagram	Pointed feature
EmreSndkc	Django skeleton, environment-driven settings (PostgreSQL/Redis/SMTP), README	System architecture	infrastructure (unpointed)
brkznqr	Model fixes: `Enrollment`, constraints, indexes, migration	ER diagram	data model of #7
eminozkanca	Calculation service, automatic recalculation (signals), 15 tests	Class diagram	#7 (5 points)
Berkegn	`seed_demo` command, Django admin panel	Role flow diagram	#2 (2 points)

Each task was done on a `feature/<name>` branch and committed under its owner's name; five branches were merged into `develop`, and `develop` into `main`. The tests (15) pass on SQLite and PostgreSQL 18.

CSE 321 — 1 Introduction

The calculation chain has three levels: assessment grade → (by percentages) Learning Outcome (LO) → (by 1–5 weights) PO. Every level uses the same formula: `score = Σ(weight × score) ÷ Σ weight`. The acceptance test is the instructor's example: for Midterm 90 and Project 80 the system produces LO1 = 86.0, LO2 = 82.9, PO1 = 85.0, PO2 = 82.9 (a unit test, and the `elif` user in the demo data).

Core feature	Status
Data model: user/role, course, enrollment, LO, PO, assessment, grade, mappings	Working
PO calculation service and precomputed PO profile (`StudentPOSnapshot`)	Working
Automatic recalculation when a grade, mapping or weight changes	Working
NULL ("no data") instead of 0 for a PO without data	Working
Django admin panel and demo data (`python manage.py seed_demo`)	Working
Role-specific screens, student dashboard, bulk grade upload, reports	Next sprints

Role	Enters	Sees
System Administrator	Users, roles	Everything
Department Head	POs, courses, instructor assignment, student enrollment	All courses and every student's PO scores
Instructor	LOs, assessments, percentages, LO→PO weights, grades	Courses, grades, PO scores
Student	—	Own grades and PO profile

Today permissions are model-level in the Django admin; the "an instructor sees only their own course" check and the student dashboard do not exist yet.

CSE 321 — 2 System Design

Architecture (Figure 1). The system is a single Django application server: a request passes through the URLs and settings under `config/` and reaches the models and the calculation service in the `outcomes/` app. External components are selected in `config/settings.py` through environment variables; if a variable is not set, a local fallback is used.

Component	Purpose	Variable	Fallback	Current state
PostgreSQL	Persistent data	`POSTGRES_DB`	SQLite	In use; tests pass on PostgreSQL 18
Redis	Cache for report queries	`REDIS_URL`	In-process memory	Configured; no query is cached yet
SMTP	E-mail notifications	`EMAIL_HOST`	Console	Configured; no feature sends e-mail yet
File storage	Excel/CSV, syllabus PDF	`MEDIA_ROOT`	`./media`	Configured; no upload screen yet

Classes (Figure 2). The code has three parts: models (`models.py`), the calculation service (`services.py`) and triggers (`signals.py`).

Class	Responsibility
`User`, `Course`, `Enrollment`, `LearningOutcome`, `ProgramOutcome`	Institutional structure: who, which course, which outcome
`Assessment`, `StudentGrade`	Assessments and raw grades
`Assessment_LO_Mapping`, `LO_PO_Mapping`	Weights: assessment → LO (coefficient), LO → PO (1–5)
`StudentPOSnapshot`	Precomputed PO profile; written only by the service
`POCalculatorService`	Grade → LO → PO calculation, snapshot writes, finding affected students
`signals`	Call the service when a grade, mapping or weight changes

The formula lives in a single function (`weighted_average`) used at both levels; when the total weight is 0 it returns `None` (no data), not 0. Example: LO1 = (0.60 × 90 + 0.40 × 80) ÷ 1.00 = 86.0; LO2 = (0.40 × 90 + 1.00 × 80) ÷ 1.40 = 82.9; PO1 = (2 × 86.0 + 1 × 82.86) ÷ 3 = 85.0; PO2 = 82.9. The instructor's 1.0 : 0.5 : 1.0 weights are entered as 2 : 1 : 2 because the model stores integer weights from 1 to 5; the result is the same.

Waffle 5.1 — System Architecture

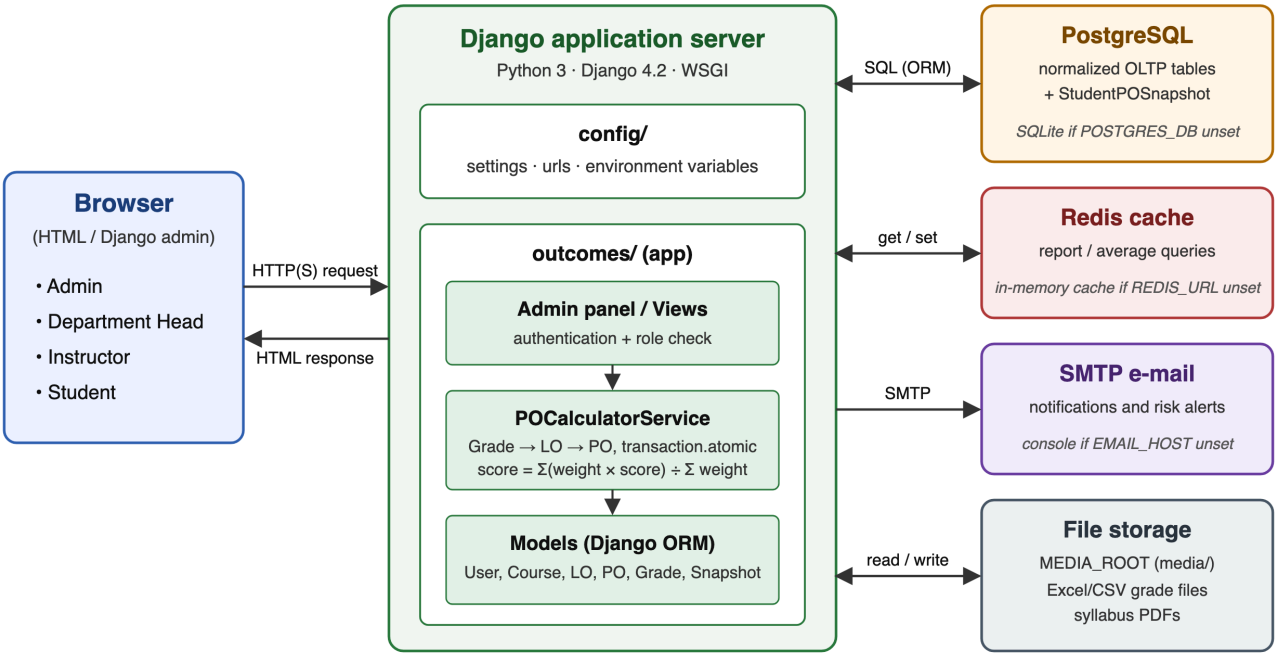
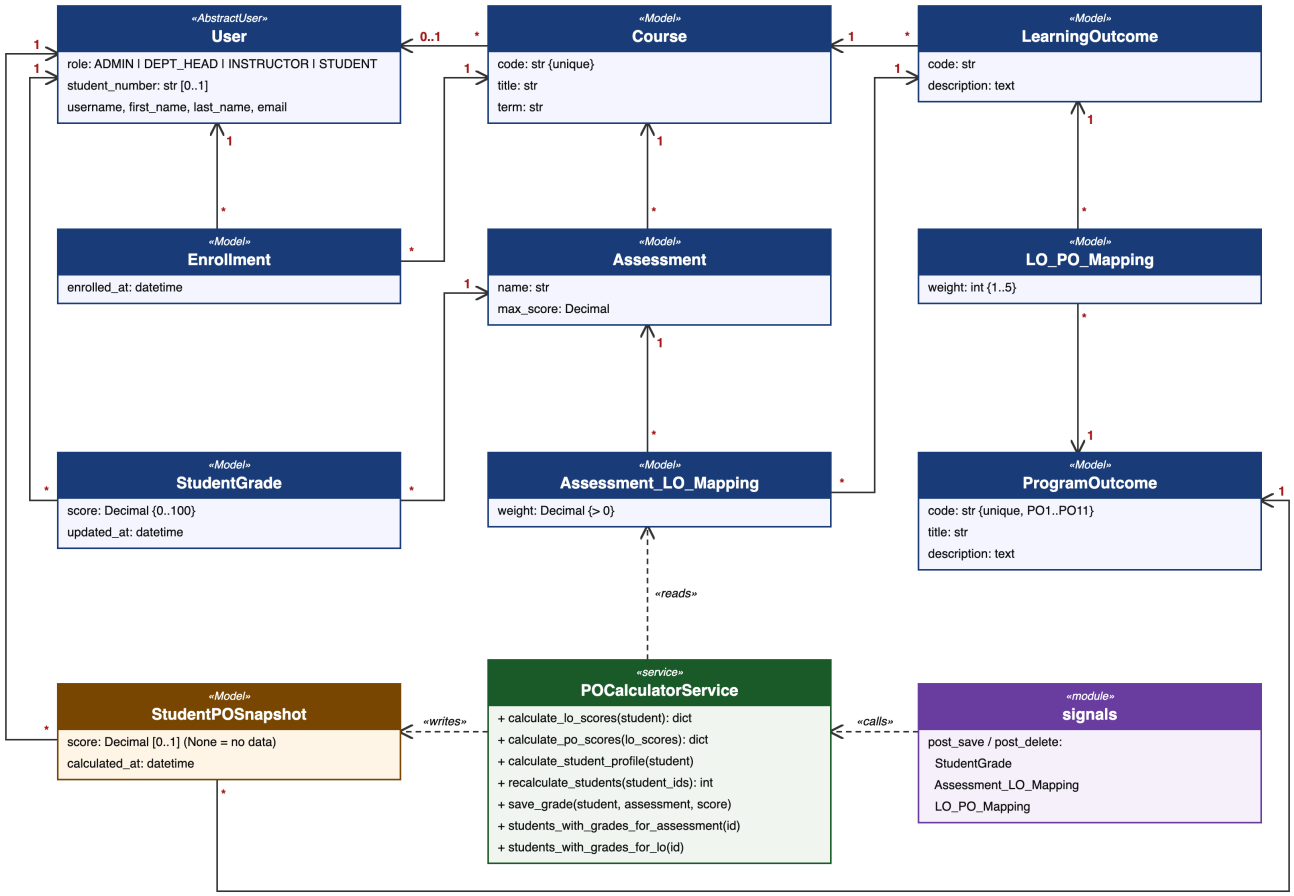


Figure 1 — System architecture



Solid arrow: association (ForeignKey), * → 1. Dashed arrow: dependency. signals call the service when a record changes; the service reads the source tables and writes the snapshot.

Figure 2 — Class diagram